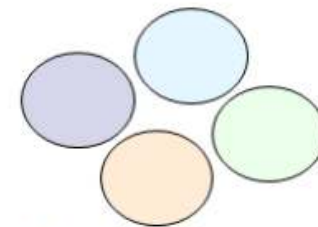


Programación Dinámica

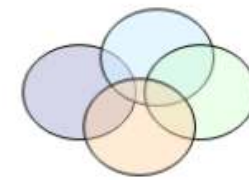
Estructura de Datos y Algoritmos II
Profesora: Elba Karen Sáenz García

Método de Programación Dinámica

- Creada por Richard Bellman en 1953.
- Normalmente es usada para resolver problemas de optimización.
- Sirve para resolver problemas combinando las soluciones de subproblemas que se traslapan y cuya solución se almacena en tablas.
- A diferencia de Divide y vencerás que puede generar subproblemas idénticos y repetición del trabajo.



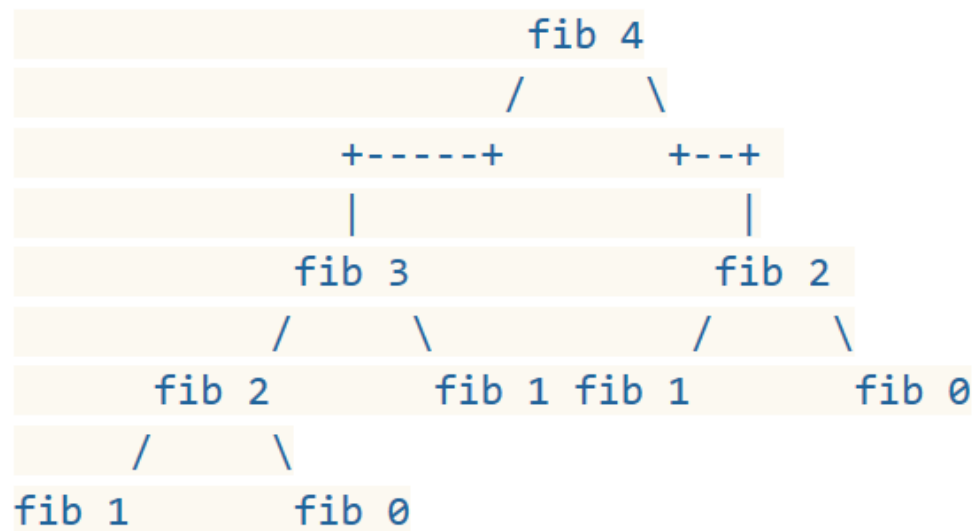
Subproblemas independientes



Subproblemas traslapados

Ejemplo

Cálculo de (fib 4) por divide y vencerás



Calcula 2 veces (fib 2) y 3 veces (fib 1) y (fib 0).

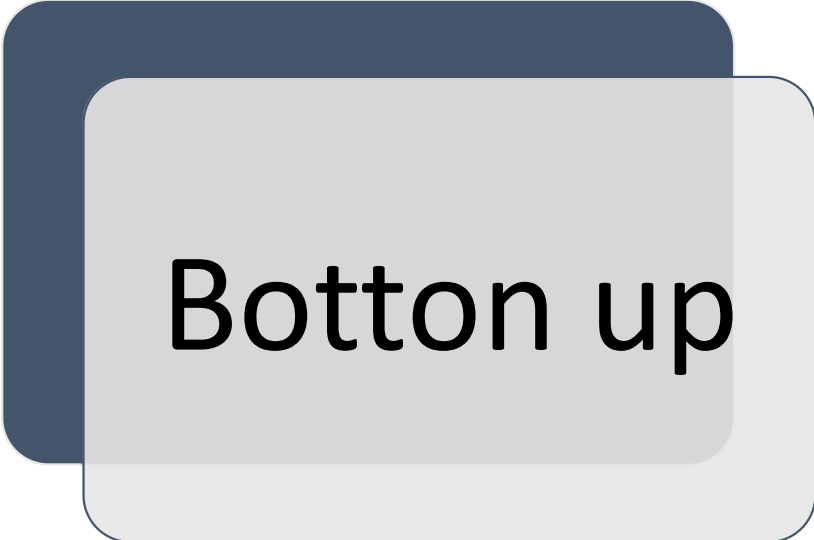
Programación Dinámica

- Los problemas se deben poder dividir en subproblemas cuya solución puede ser utilizada varias veces (se traslapan).
- Conserva la solución a cada sub-problema que ya se ha resuelto en una tabla, para tomarla cuando ésta se requiera.
- Evitar el solapamiento, es decir, evitar calcular varias veces un mismo resultado.
- Esto reduce el tiempo necesario para obtener el resultado final, pues evita repetir algunos de los cálculos.
- Se hace uso de la recursión para diseño.

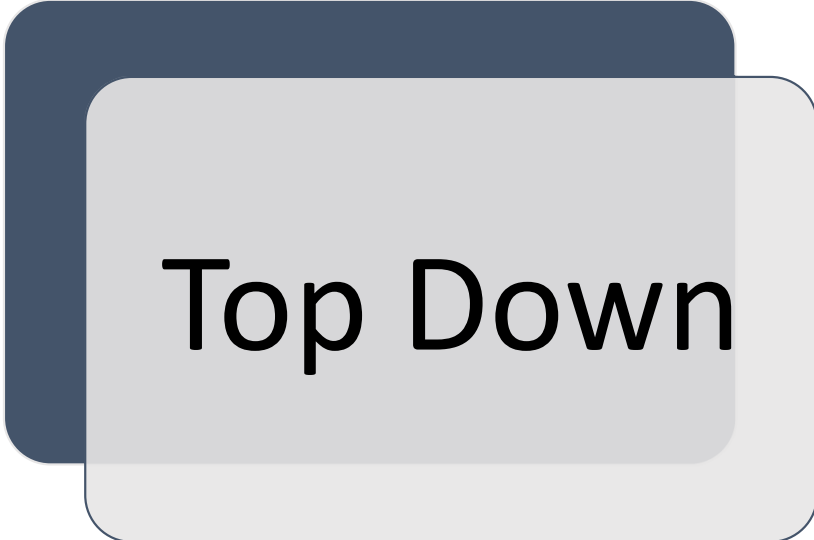
Los pasos para el diseño de un algoritmo de programación dinámica:

- 1.- Plantear un algoritmo de solución recursivo.
- 2.- Organizar las soluciones parciales en una tabla.
- 3.- Modificar el algoritmo recursivo de manera que antes de hacer el llamado recursivo se consulte la tabla (memonización) y, en caso de hallarse la solución en ésta, ya no se hace el llamado recursivo.

Dos esquemas

A diagram illustrating the 'Bottom up' schema. It consists of two overlapping rounded rectangles. The front rectangle is light gray and contains the text 'Bottom up'. The back rectangle is dark blue and is partially obscured by the front one.

Bottom up

A diagram illustrating the 'Top Down' schema. It consists of two overlapping rounded rectangles. The front rectangle is light gray and contains the text 'Top Down'. The back rectangle is dark blue and is partially obscured by the front one.

Top Down

Ejemplo Botton up Cálculo de los números de Fibonacci

La expresión recursiva es:

$$F(n) = \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F(n - 1) + F(n - 2) & \text{if } n > 1. \end{cases}$$

Plantear un algoritmo de solución recursivo.
Pseudocódigo

FibonacciRecursivo(n)

Inicio

Si $n \leq 1$

retornar n

Si no

retornar (FibonacciRecursivo(n-1)+ FibonacciRecursivo(n-2))

Fin Si

FIN

Organizar las soluciones parciales en una tabla.

- Generando una tabla para ir guardando los resultados parciales
- En la tabla se almacenan inicialmente los casos base.
- El algoritmo será mucho más eficiente, pues en lugar de hacer dos llamados recursivos solo sacarán dos resultados de la tabla cuando estos ya han sido calculados.

Ejemplo números de fibonacci

$$F(n) = \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F(n-1) + F(n-2) & \text{if } n > 1. \end{cases}$$

F ₀	F ₁	F ₂	F ₃	F ₄	F ₅	F ₆	F ₇	F ₈	F ₉	F ₁₀	F ₁₁	F ₁₂	F ₁₃	F ₁₄	F ₁₅	F ₁₆	F ₁₇	F ₁₈	F ₁₉	F ₂₀
0	1	1	2	3	5	8	13	21	34	55	89	144	233	377	610	987	1597	2584	4181	6765

Fibonacci con programación Dinámica Botton up

- Se resuelven todas las instancias a partir de las mas chicas hasta la que se desea calcular

$F[0]=0$

$F[1]=1$

Para $i=2$ hasta n

$$F[i]=F[i-1]+F[i-2]$$

- Las soluciones se irán almacenando para después utilizarlas

¿Cómo implementar en Python?

$F[0]=0$

$F[1]=1$

Para $i=2$ hasta n

$F[i]=F[i-1]+F[i-2]$

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}	F_{16}	F_{17}	F_{18}	F_{19}	F_{20}
0	1	1	2	3	5	8	13	21	34	55	89	144	233	377	610	987	1597	2584	4181	6765

Programación Dinámica -Top Down

- A diferencia de bottom-up, se empiezan a hacer los cálculos de n hacia abajo.
- Divide el problema en subproblemas
- Si el subproblema ya se resolvió reutiliza la respuesta
- De otra manera, resuelve el subproblema y lo almacena
- Utiliza la técnica llamada *memoización* la cual consiste en guardar los resultados previamente calculados, de tal manera que no se tengan que repetir operaciones.

FiboDina(n)

Inicio

Si $n==0$ o $n==1$

$T[n]=n$

Fin Si

Si $Tabla[n]==0$

$Tabla[n]=FiboDina(n-1)+ FiboDina(n-2)$

Retorna $Tabla[n]$

Fin

FibRecursivo(n)

Inicio

Si $n \leq 1$

 retornar n

Si no

 retornar $(FibRecursivo(n-1)+ FibRecursivo(n-2))$

Fin Si

FIN

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}	F_{16}	F_{17}	F_{18}	F_{19}	F_{20}
0	1	1	2	3	5	8	13	21	34	55	89	144	233	377	610	987	1597	2584	4181	6765

¿Cómo se realiza en Python?

DV vs Programación Dinámica

- Similitud
 - Descomposición recursiva del problema
- Diferencias
 - DV aplica directamente la fórmula recursiva
 - PD- Resolver primero los problemas mas pequeños guardando los datos en una tabla (iterativo)

DV vs Programación Dinámica

Similitud

- Descomposición recursiva del problema

Diferencias

- DV aplica directamente la fórmula recursiva
- PD- Resolver primero los problemas más pequeños guardando los datos en una tabla (iterativo)

Ejemplo Problema de la mochila 0/1

- Los objetos no se pueden partir, se escogen enteros o no se escogen

Datos

N = número de objetos

W =Capacidad de la mochila

P=(p1,p2,.... , pn)

V=(v1,v2,....vn)

Minimizar

sujeto a

$$\sum_{i=1}^n v_i x_i$$

$$\sum_{i=1}^n p_i x_i \leq W \quad x \in \{0,1\}$$

Una forma de realizar el diseño de un algoritmo de programación dinámica:

- 1.- Plantear un algoritmo de solución recursivo.
- 2.- Organizar las soluciones parciales en una tabla.
- 3.- Modificar el algoritmo recursivo de manera que antes de hacer el llamado recursivo se consulte la tabla y, en caso de hallarse la solución en ésta, ya no se hace el llamado recursivo.

Paso 1 plantear subproblemas y recurrencia

Interpretar el problema como proceso toma de decisiones

Si **se escoge** un objeto k se tiene el valor v_k pero , pero en la mochila queda menos espacio

Si **no se escoge** se tiene el mismo problema pero con un objeto menos



Después de tomar una decisión sobre un objeto queda un problema de menor tamaño , con un objeto menos

- Mochila(k,m)

K objetos

m capacidad

Regresa el valor total

Casos base

- Si $m=0$ no se puede incluir objetos

Mochila (k,0)=0

- Si $k=0$ no se incluye

Mochila (0,m)=0

-Si m y k son negativos el problema es irresoluble

Mochila(k,m)= -infinito

$$\text{Mochila}(k, m) = \begin{cases} 0 & \text{Si } k=0 \text{ o } m=0 \\ -\infty & \text{Si } k<0 \text{ o } m<0 \\ \max \{ \text{Mochila}(k-1, m), v_k + \text{Mochila}(k-1, m-p_k) \} & \end{cases}$$

- Para aplicar programación dinámica de forma ascendente (bottom up) se crea una tabla V de $n \times W$ para que $Mochila(i,j) = V[i][j]$
- Ejemplo: n objetos = 3 $W = 6$ $p = (2,3,4)$ y $v = (1,2,5)$

Se llenan los casos base

		j						
	V^w	0	1	2	3	4	5	6
i	0	0	0	0	0	0	0	0
	1	0						
	2	0						
	3	0						

Para aplicar programación dinámica de forma ascendente (bottom up) se crea una tabla V de n X W para que $Mochila(i,j)=V[i][j]$

$$Mochila(k, m) = \begin{cases} 0 & \text{Si } k=0 \text{ o } m=0 \\ -\infty & \text{Si } k<0 \text{ o } m<0 \\ \max \{Mochila(k-1, m), v_k + Mochila(k-1, m-p_k)\} & \end{cases}$$

Forma de llenar tablas

Colocar en tabla valores de los casos base

$$V[i][0]=0$$

$$V[0][j]=0$$

Para i=1 hasta n

Para j=1 hasta W

$$V[i][j] = \max(V[i-1][j], v[i] + V[i-1][j-p[i]])$$

Ojo: si $j-p_i$ es negativo, entonces es el caso $-\infty$, y el máximo será siempre el otro término.

El valor Optimo esta en $V[n][W]$

Problema de la Mochila0/1

- Ejemplo. $n=3$, $M=6$, $p=(2, 3, 4)$, $v=(1, 2, 5)$

		j						
i	V_M	0	1	2	3	4	5	6
	0	0	0	0	0	0	0	0
	1	0	0	1	1	1	1	1
	2	0	0	1	2	2	3	3
	3	0	0	1	2	5	5	6

Forma de llenar tablas

$V[i][0]=0$

$V[0][j]=0$

Para $i=1$ hasta n

Para $j=1$ hasta W

$V[i][j] = \max(V[i-1][j], v[i]+V[i-1][j - p[i]])$

El valor Optimo esta en $V[n][W]$

Ojo: si $j-p_i$ es negativo, entonces es el caso $-\infty$, y el máximo será siempre el otro término.

Componer solución optima

En $V[n][W]$ almacena el valor optimo

Pero, ¿Cuáles objetos se escogen?

Se obtiene la tupla $(x_1, x_2, x_3, \dots, x_n)$ usando la Tabla V

Idea

-Partiendo de la posición $V[n][W]$

Se analizan las decisiones que se tomaron para cada objeto i

a) Si $V[i][j] = V[i-1][j]$ entonces la solución no usa el objeto i

$$x_i = 0$$

b) Si $V[i][j] = v[i] + V[i-1][j - p[i]]$, entonces se usa el objeto i

$$x_i = 1$$

c) Si se cumplen ambas, entonces podemos usar el objeto i o no

- Ejemplo. $n = 3$, $M = 6$, $p = (2, 3, 4)$, $v = (1, 2, 5)$

		j						
i	V_M	0	1	2	3	4	5	6
	0	0	0	0	0	0	0	0
	1	0	0	1	1	1	1	1
	2	0	0	1	2	2	3	3
	3	0	0	1	2	5	5	6

Una forma de componer la solución

- Ejemplo. $n = 3$, $M = 6$, $p = (2, 3, 4)$, $v = (1, 2, 5)$

$j = W$

Para $i = n$ hasta 1

 si $V[i][j] == V[i-1][j]$

$x[i] = 0$

 Sino

$x[i] = 1$

$j = j - p[i]$

 fin i

Fin para

		j						
i	V_M	0	1	2	3	4	5	6
	0	0	0	0	0	0	0	0
	1	0	0	1	1	1	1	1
	2	0	0	1	2	2	3	3
	3	0	0	1	2	5	5	6

```
def mochila():
    V=creaTabla(3,6)
```

```
    v=[1,2,5]
```

```
    p=[2,3,4]
```

```
    n=3
```

```
    W=6
```

```
    x=[0 for i in range(n)]
```

```
    print (x)
```

```
    llenaTabla(V,3,W,v,p)
```

```
    obtenerSol(V,p,W,n,x)
```

```
    print (V)
```

```
    print(x)
```

```
mochila()
```

```
[0, 0, 0]
```

```
[[0, 0, 0, 0, 0, 0, 0], [0, 0, 1, 1, 1, 1, 1], [0, 0, 1, 2, 2, 3, 3], [0, 0, 1, 2, 5, 5, 6]]
```

```
[1, 0, 1]
```

- Ejemplo. $n=3$, $M=6$, $p=(2, 3, 4)$, $v=(1, 2, 5)$

		j							
		V _M	0	1	2	3	4	5	6
i	0	0	0	0	0	0	0	0	0
	1	0	0	1	1	1	1	1	1
	2	0	0	1	2	2	3	3	3
	3	0	0	1	2	5	5	6	6

Ejercicio

Programa en Python que permita indicar de la entrada estándar n, W , el valor de los objetos $v[]$ y su peso $p[]$. Probar con el siguiente ejemplo

Ejemplo

Mochila de tamaño **$W=11$**

Número de objetos **$n=5$**

Solución óptima **$\{3,4\}$**

Objeto	Valor	Peso
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

	0	1	2	3	4	5	6	7	8	9	10	11
ϕ	0	0	0	0	0	0	0	0	0	0	0	0
$\{1\}$	0	1	1	1	1	1	1	1	1	1	1	1
$\{1, 2\}$	0	1	6	7	7	7	7	7	7	7	7	7
$\{1, 2, 3\}$	0	1	6	7	7	18	19	24	25	25	25	25
$\{1, 2, 3, 4\}$	0	1	6	7	7	18	22	24	28	29	29	40
$\{1, 2, 3, 4, 5\}$	0	1	6	7	7	18	22	28	29	34	35	40

Eloa Karen Sáenz García

Otros ejemplos

- **Producto de cadenas de matrices**
- **Camino mínimos entre todos los pares de nodos de un grafo**
- **Problema del viajero**
- **Problema del cambio**

Backtracking

Estructuras de Datos y Algoritmos I
Profesora Elba Karen Sáenz García

Backtracking: Vuelta atrás (retroceso)

- Se aplica a problemas que requieren de una búsqueda exhaustiva dentro del conjunto de todas las soluciones potenciales.
- El espacio a explorar se estructura de tal forma que se puedan ir descartando bloques de soluciones que posiblemente no son satisfactorias.
- En caso de llegar a una etapa la cual no puede ser solución del problema, retrocede a la etapa anterior y toma un camino diferente en busca de una solución, en caso de que tampoco sea una solución, retrocede y toma otro camino, así sucesivamente.
- De esta manera nos garantiza que encontrará todas o al menos una de las soluciones posibles al problema.

Backtracking: Vuelta atrás (retroceso)

Para lograr esto, los algoritmos de tipo *backtracking* construyen posibles soluciones candidatas de manera sistemática.

En general, dado una solución candidata S :

- Se verifica si S es solución. Si lo es hace algo con ella (depende del problema)
- Construyen todas las posibles extensiones de S , e invocan recursivamente al algoritmo con todas ellas.

Componentes

- Se trabaja entonces con:
 - Una **solución** que se puede expresar como una tupla, $(x_1, x_2, \dots, x_n)$, que satisface unas restricciones y tal vez que optimice cierta función objetivo.
 - **Espacio de Soluciones:** Conjunto de soluciones que se puede representar mediante un árbol
 - **Solución Parcial** .En cada momento, el algoritmo se encontrará en cierto nivel **k**, con una solución parcial $(x_1, \dots, x_k)$ en la que todavía no se han definido todos sus componentes.
 - **Función Poda o acotación** la cual permite identificar cuando una solución parcial no conduce a la solución

Método General

En cada momento, el algoritmo se encontrará en cierto nivel **k**, **con una solución parcial ($x_1, \dots, x_k$)**.

Si se puede añadir un nuevo elemento a la solución **x_{k+1}** , **se genera y se avanza al nivel $k+1$** .

Si no, se prueban otros valores para **x_k** . **(Mismo algoritmo Hay recursividad)**

Si no existe ningún valor posible por probar, entonces se retrocede al nivel anterior **$k-1$** .

Se sigue hasta que la solución parcial sea una solución completa del problema, o hasta que no queden más posibilidades por probar.

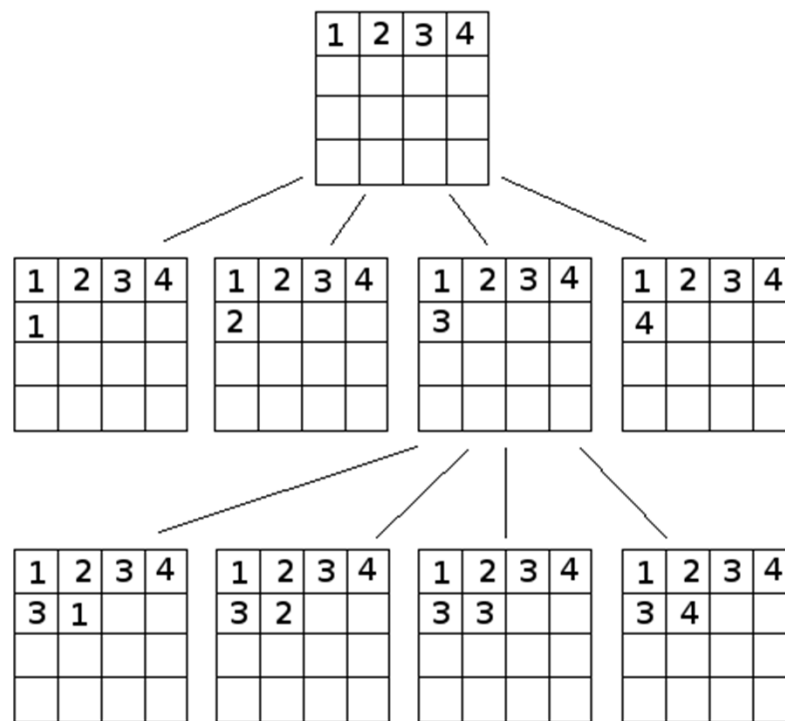
- Esquema para todas las soluciones:

```
Procedimiento Backtracking (Tipoetapa etapa)
  para cada candidato hacer
    | seleccionar candidato
    | si aceptable entonces
    | inicio
    |   anotar_candidato
    |   si solucion_incompleta entonces
    |     Backtracking (etapa_siguiente)
    |   sino
    |     almacenar_solucion
    |     borrar_candidato
    | fin
  hasta que candidatos_agotados
fin procedimiento
```

Ejemplos

Salida en un laberinto,
Resolver sudokus dinámicamente,
El problema de las n reinas,
El problema de la mochila.
Problema del cambio

Sudoku



Ejemplo Problema n reinas

Restricciones explícitas:

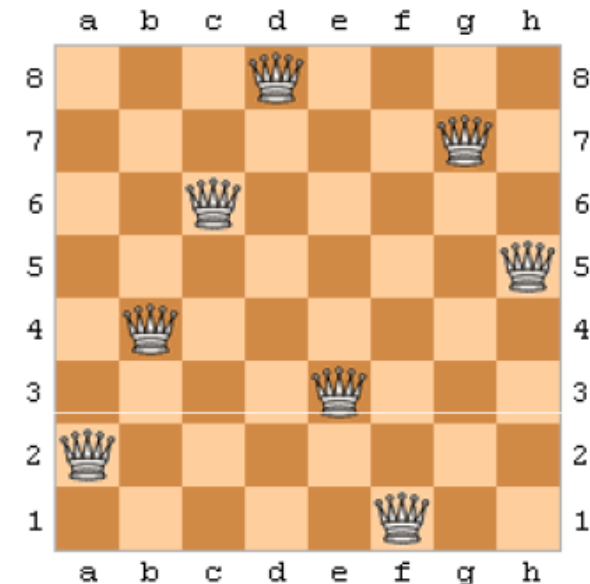
$$S_i = \{1, 2, 3, 4, 5, 6, 7, 8\}, 1 \leq i \leq 8$$

Espacio de soluciones:

$$\text{Tamaño } |S_i|^8 = 8^8 = 2^{24} = 16M$$

Restricciones implícitas:

- Ningún par (x_i, x_j) con $x_i = x_j$
(todas las reinas deben estar en columnas diferentes).
- Ningún par (x_i, x_j) con $|j-i| = |x_j - x_i|$
(todas las reinas deben estar en diagonales diferentes).



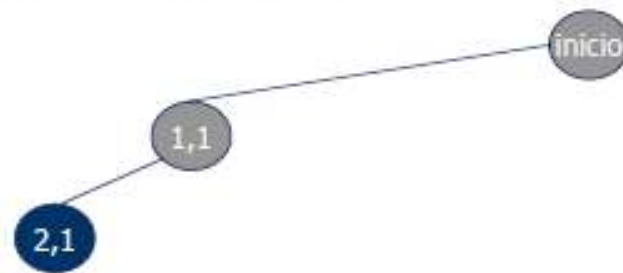
Generación de estados

Generación de estados usando backtracking



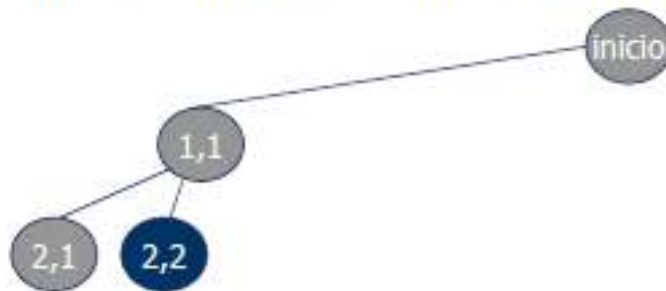
OK! Adelante con la búsqueda...

Generación de estados usando backtracking



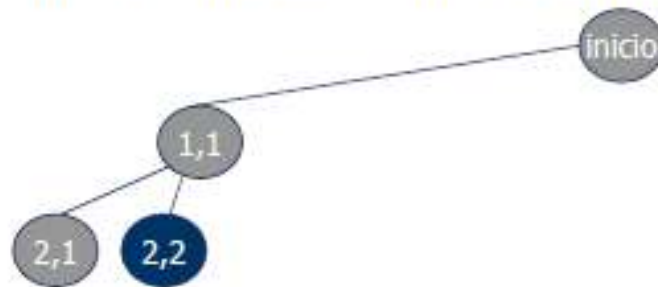
No se cumplen las restricciones: Misma columna.

Generación de estados usando backtracking



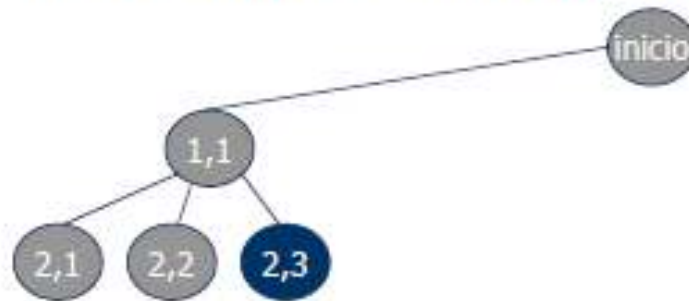
No se cumplen las restricciones: Misma diagonal.

Generación de estados usando backtracking



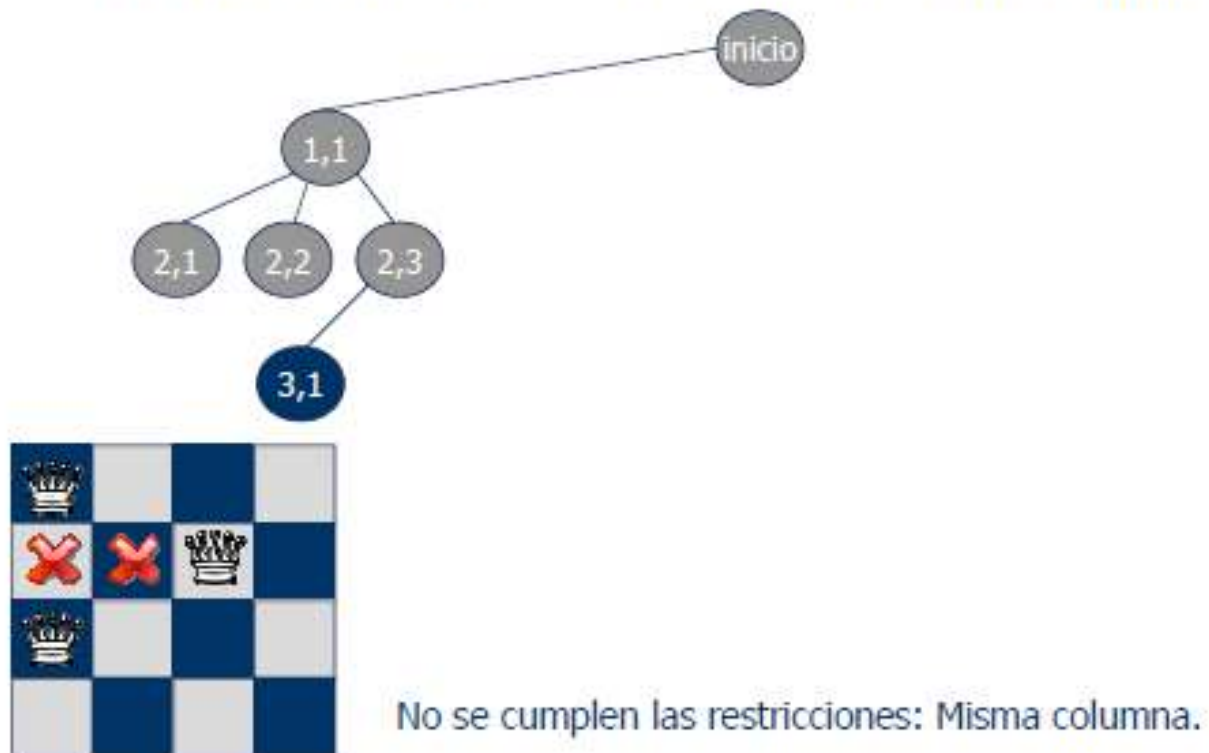
No se cumplen las restricciones: Misma diagonal.

Generación de estados usando backtracking



OK! Adelante con la búsqueda...

Generación de estados usando backtracking



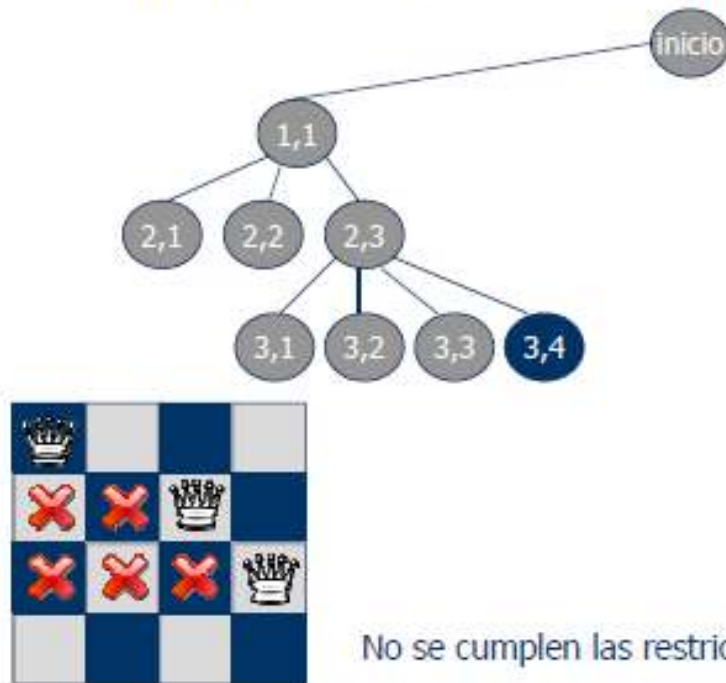
Generación de estados usando backtracking



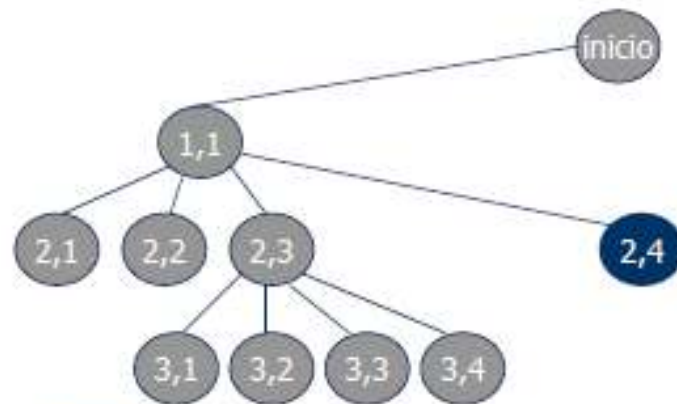
Generación de estados usando backtracking



Generación de estados usando backtracking

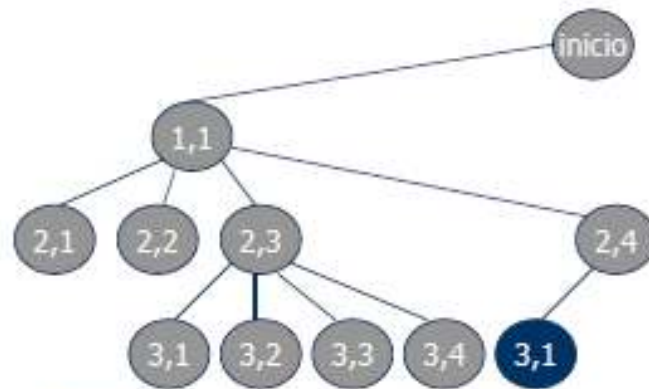


Generación de estados usando backtracking



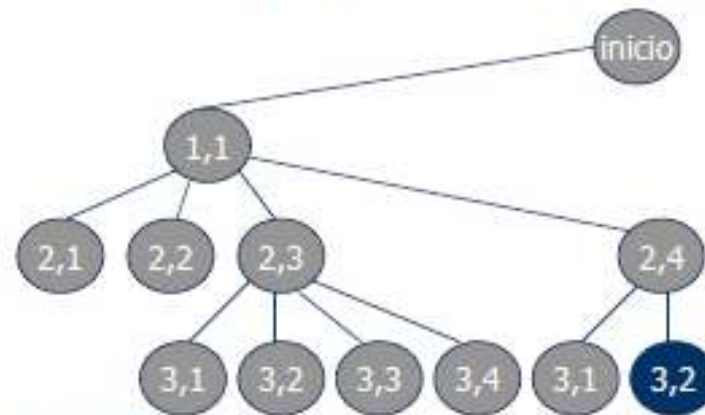
OK! Adelante con la búsqueda...

Generación de estados usando backtracking



No cumple las restricciones: Misma columna.

Generación de estados usando backtracking

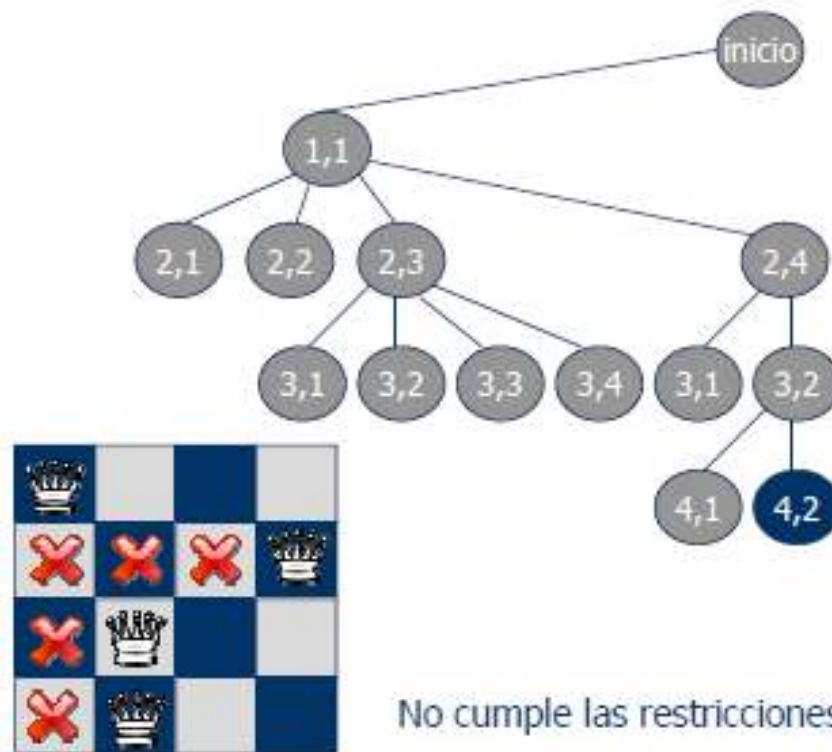


OK! Adelante con la búsqueda...

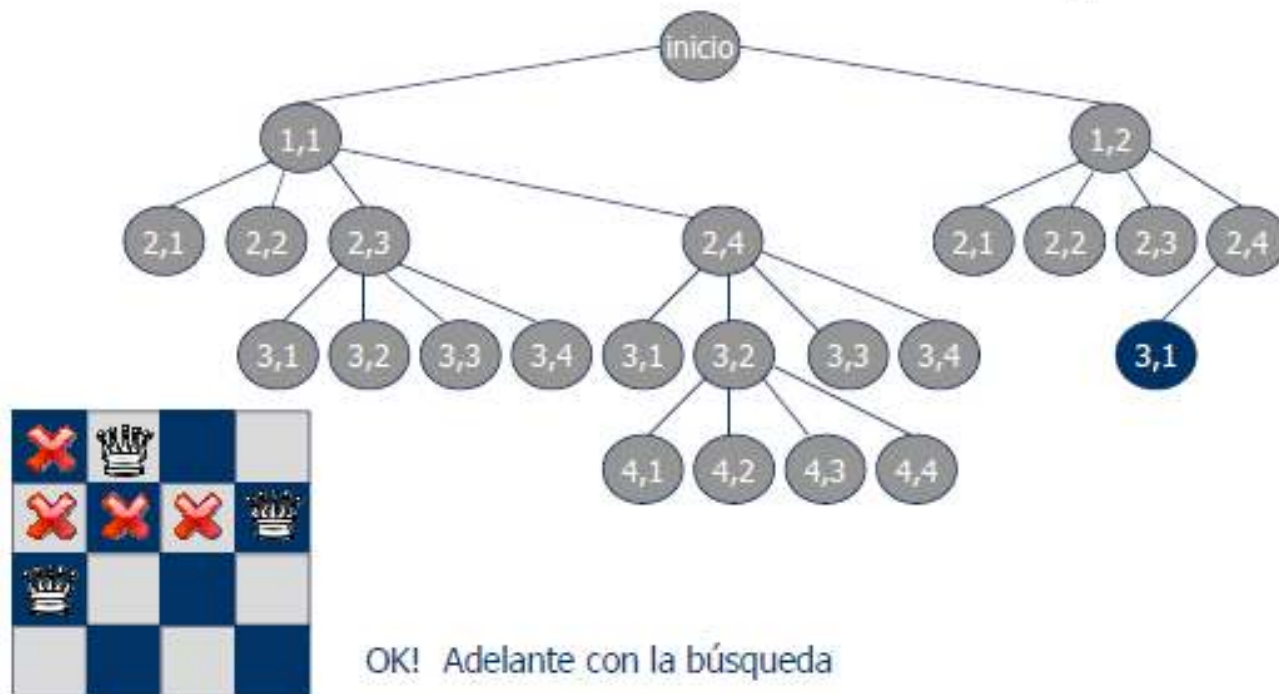
Generación de estados usando backtracking



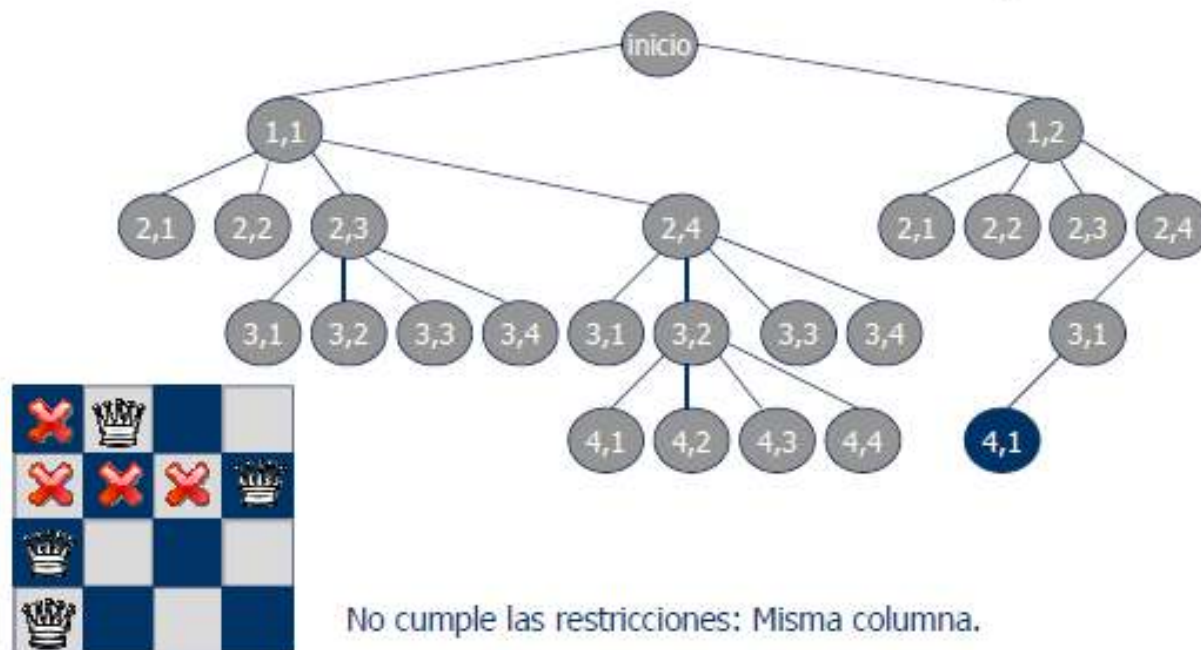
Generación de estados usando backtracking



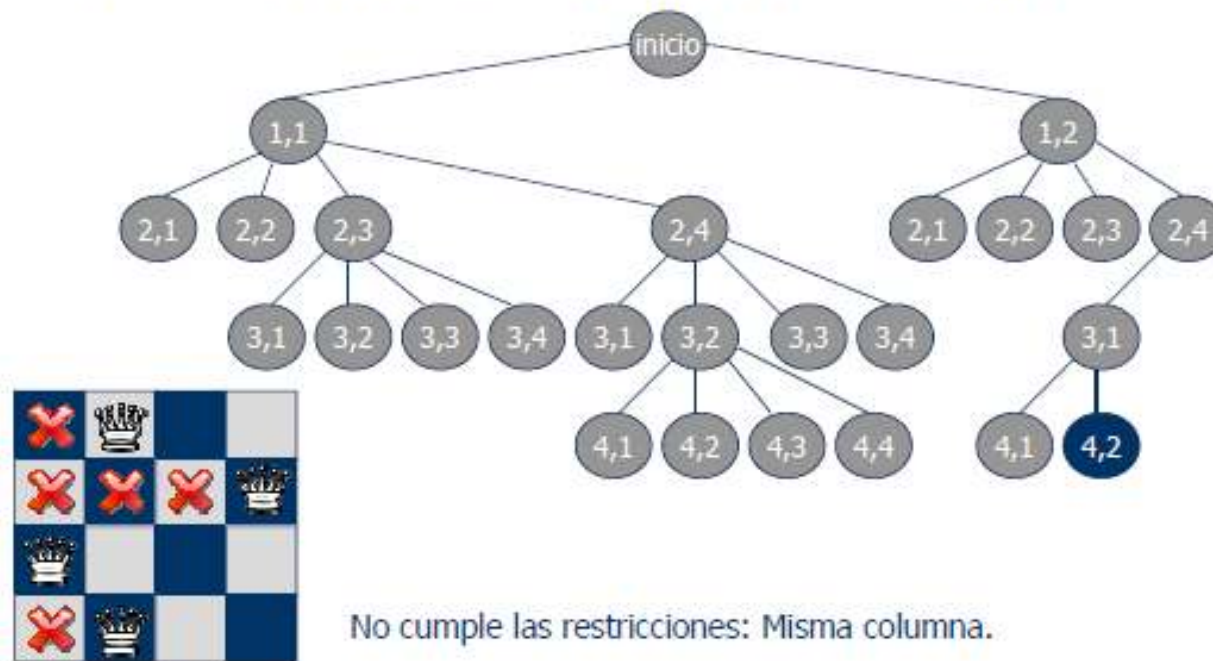
Generación de estados usando backtracking



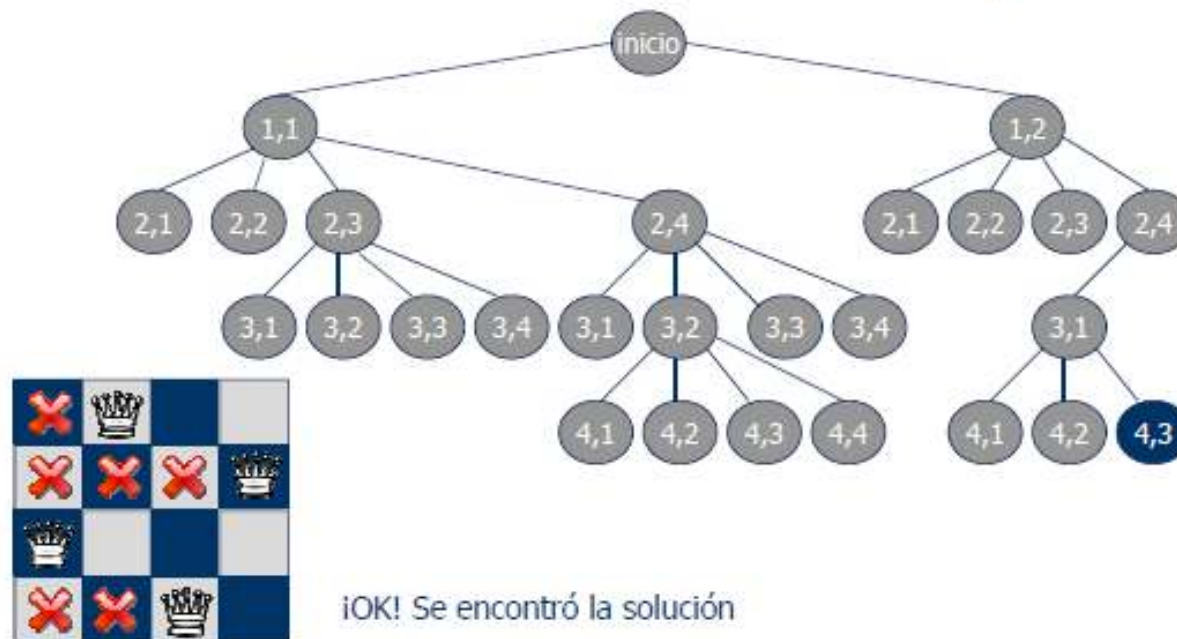
Generación de estados usando backtracking



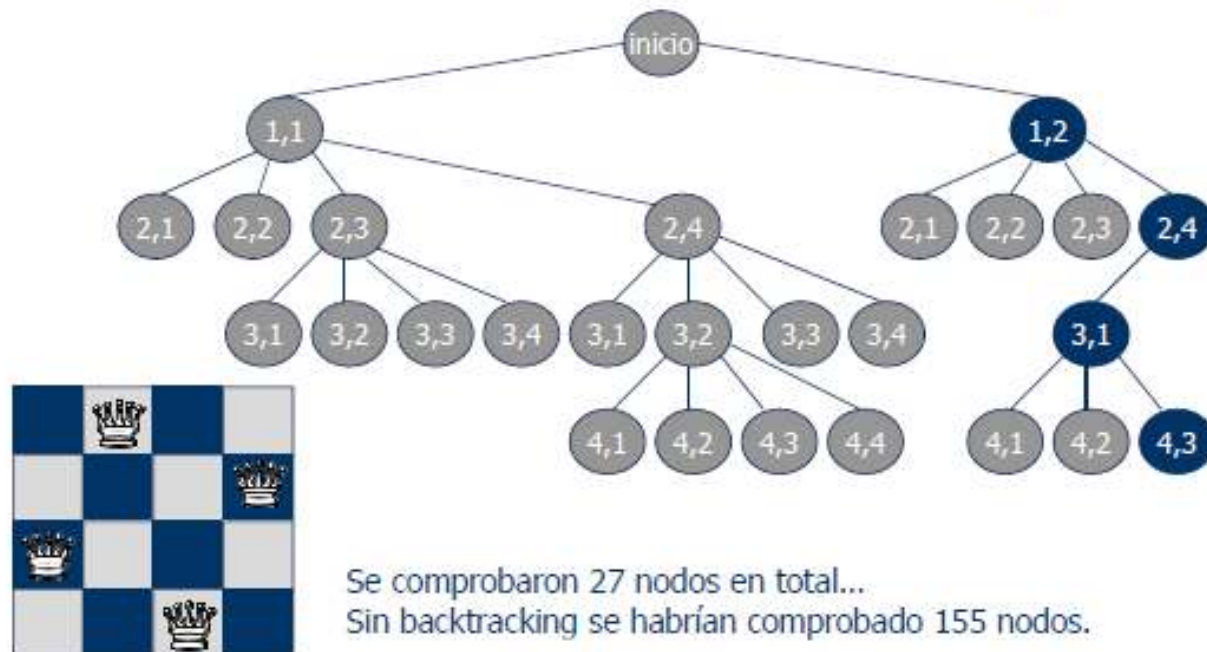
Generación de estados usando backtracking



Generación de estados usando backtracking



Generación de estados usando backtracking



columna	4	2	5	8	6	1	3	7
fila	1	2	3	4	5	6	7	8



Para saber la solución en un arreglo, considerar la validez
Las reinas en las posiciones

No se atacan entre si

$$(1, A[1]), (2, A[2]), \dots, (k, A[k])$$

Restricciones

- Considerar las restricciones

$$A[i] \neq A[j] \quad i \neq j \quad i, j \in [1, k]$$

$$|A[i] - A[j]| \neq |i - j| \quad i \neq j \quad i, j \in [1, k]$$

- La primera nos dice que no puede haber un par de reinas en la misma columna
- La segunda, que no puede haber un par de reinas en la misma diagonal

columna	4	2	5	8	6	1	3	7
fila	1	2	3	4	5	6	7	8



REINAS(A,k,n)

if $k = n$

then if PROMETEDOR(A,k)

▷ Se encontró la solución!

then print A

else return

else

for $j \leftarrow 1$ **to** n

do if PROMETEDOR(A,k)

then $A[k+1] \leftarrow j$

REINAS(A,k+1,n)

PROMETEDOR(A,k)

for $j \leftarrow 1$ **to** $k-1$

do if $A[j] = A[k]$ or $|A[j] - A[k]| = |j - k|$

return FALSE

return TRUE